Lab 10 – Programs Using Different CUDA Device Memory Types and Synchronization

Q1.cu

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

__global__ void matrixMulKernel(float* A, float* B, float* C, int N) {
int row = blockIdx.y * blockDim.y + threadIdx.y;
int col = blockIdx.x * blockDim.x + threadIdx.x;

if (row < N && col < N) {
float value = 0;
for (int k = 0; k < N; k++) {
value += A[row * N + k] * B[k * N + col];
}
C[row * N + col] = value;
}
}

int main() {
int N;
printf("Enter the size of the square matrix (N): ");
scanf("%d", &N);

size_t size = N * N * sizeof(float);

float *h_A = (float*)malloc(size);
float *h_B = (float*)malloc(size);
float *h_C = (float*)malloc(size);

printf("Enter elements for Matrix A (%dx%d):\n", N, N);
for (int i = 0; i < N * N; i++) {
scanf("%f", &h_A[i]);
}

printf("Enter elements for Matrix B (%dx%d):\n", N, N);
for (int i = 0; i < N * N; i++) {
scanf("%f", &h_B[i]);
}
```

```
float *d_A, *d_B, *d_C;
cudaMalloc((void**)&d_A, size);
cudaMalloc((void**)&d_B, size);
cudaMalloc((void**)&d_C, size);

cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

dim3 threadsPerBlock;
threadsPerBlock.x = 16;
threadsPerBlock.y = 16;
threadsPerBlock.z = 1;

dim3 blocksPerGrid;
blocksPerGrid.x = (N + threadsPerBlock.x - 1) / threadsPerBlock.x;
blocksPerGrid.y = (N + threadsPerBlock.y - 1) / threadsPerBlock.y;
blocksPerGrid.z = 1;

matrixMulKernel<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, d_C, N);

cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

printf("\nResulting Matrix C:\n");
for (int i = 0; i < N; i++) {
for (int j = 0; j < N; j++) {
printf("%.2f ", h_C[i * N + j]);
}
printf("\n");
}

cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);
free(h_A);
free(h_B);
free(h_C);

return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L10$ nvcc -o abc q1.cu
6CSEC1@debian:~/Documents/230905152_PPL/L10$ ./abc
Enter the size of the square matrix (N): 3
Enter elements for Matrix A (3x3):
1 2 3
1 2 3
1 2 3
Enter elements for Matrix B (3x3):
3 2 1
3 2 1
3 2 1

Resulting Matrix C:
18.00 12.00 6.00
18.00 12.00 6.00
18.00 12.00 6.00
```

Q2.cu

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define MAX_MASK_SIZE 1024
__constant__ float d_mask[MAX_MASK_SIZE];

__global__ void convolution1D(float* I, float* J, int n, int mask_width) {
int i = blockIdx.x * blockDim.x + threadIdx.x;
if (i < n) {
float result = 0.0f;
int radius = mask_width / 2;

for (int j = 0; j < mask_width; j++) {
int cur_index = i + j - radius;
if (cur_index >= 0 && cur_index < n) {
result += I[cur_index] * d_mask[j];
}
}
J[i] = result;
}
}

int main() {
```

```
int n, m_width;

printf("Enter size of input array (n): ");
scanf("%d", &n);

printf("Enter size of mask (must be odd and <= 1024): ");
scanf("%d", &m_width);

size_t size_I = n * sizeof(float);
size_t size_M = m_width * sizeof(float);

float *h_I = (float*)malloc(size_I);
float *h_J = (float*)malloc(size_I);
float *h_M = (float*)malloc(size_M);

printf("Enter %d elements for input array:\n", n);
for (int i = 0; i < n; i++) scanf("%f", &h_I[i]);

printf("Enter %d elements for mask:\n", m_width);
for (int i = 0; i < m_width; i++) scanf("%f", &h_M[i]);

float *d_I, *d_J;
cudaMalloc((void**)&d_I, size_I);
cudaMalloc((void**)&d_J, size_I);

cudaMemcpy(d_I, h_I, size_I, cudaMemcpyHostToDevice);
cudaMemcpyToSymbol(d_mask, h_M, size_M);

int threads = 256;
int blocks = (n + threads - 1) / threads;

convolution1D<<<blocks, threads>>>(d_I, d_J, n, m_width);

cudaMemcpy(h_J, d_J, size_I, cudaMemcpyDeviceToHost);

printf("\nResultant Array:\n");
for (int i = 0; i < n; i++) {
printf("%.2f ", h_J[i]);
}
printf("\n");

cudaFree(d_I);
cudaFree(d_J);
```

```
free(h_I);
free(h_J);
free(h_M);

return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L10$ nvcc -o abc q2.cu
6CSEC1@debian:~/Documents/230905152_PPL/L10$ ./abc
Enter size of input array (n): 5
Enter size of mask (must be odd and <= 1024): 3
Enter 5 elements for input array:
10 20 30 50 50
Enter 3 elements for mask:
2 2 2

Resultant Array:
60.00 120.00 200.00 260.00 200.00
```

Q3.cu

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

__global__ void tiledConvolution1D(float* N, float* P, float* M, int width, int mask_width) {
extern __shared__ float N_ds[];

int i = blockIdx.x * blockDim.x + threadIdx.x;
int radius = mask_width / 2;
int l_idx = threadIdx.x + radius;
if (i < width) {
N_ds[l_idx] = N[i];
} else {
N_ds[l_idx] = 0.0f;
}

if (threadIdx.x < radius) {
int left_neighbor = i - radius;
N_ds[threadIdx.x] = (left_neighbor >= 0) ? N[left_neighbor] : 0.0f;
}

if (threadIdx.x >= blockDim.x - radius) {
```

```
int right_neighbor = i + radius;
if (right_neighbor < width) {
N_ds[l_idx + radius] = N[right_neighbor];
} else {
N_ds[l_idx + radius] = 0.0f;
}
}

__syncthreads();

if (i < width) {
float Pvalue = 0.0f;
for (int j = 0; j < mask_width; j++) {
Pvalue += N_ds[threadIdx.x + j] * M[j];
}
P[i] = Pvalue;
}
}

int main() {
int width, m_width;
printf("Enter array width: ");
scanf("%d", &width);
printf("Enter mask width (must be odd): ");
scanf("%d", &m_width);

size_t size_N = width * sizeof(float);
size_t size_M = m_width * sizeof(float);

float *h_N = (float*)malloc(size_N);
float *h_P = (float*)malloc(size_N);
float *h_M = (float*)malloc(size_M);

printf("Enter %d elements for array N: ", width);
for (int i = 0; i < width; i++) scanf("%f", &h_N[i]);
printf("Enter %d elements for mask M: ", m_width);
for (int i = 0; i < m_width; i++) scanf("%f", &h_M[i]);

float *d_N, *d_P, *d_M;
cudaMalloc((void**)&d_N, size_N);
cudaMalloc((void**)&d_P, size_N);
cudaMalloc((void**)&d_M, size_M);

cudaMemcpy(d_N, h_N, size_N, cudaMemcpyHostToDevice);
```

```
cudaMemcpy(d_M, h_M, size_M, cudaMemcpyHostToDevice);

int threads = 16;
int blocks = (width + threads - 1) / threads;
size_t sharedMemBytes = (threads + m_width - 1) * sizeof(float);

tiledConvolution1D<<<blocks, threads, sharedMemBytes>>>(d_N, d_P, d_M, width, m_width);

cudaMemcpy(h_P, d_P, size_N, cudaMemcpyDeviceToHost);

printf("\nResult P:\n");
for (int i = 0; i < width; i++) printf("%.2f ", h_P[i]);
printf("\n");

cudaFree(d_N); cudaFree(d_P); cudaFree(d_M);
free(h_N); free(h_P); free(h_M);

return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L10$ nvcc -o abc q3.cu
6CSEC1@debian:~/Documents/230905152_PPL/L10$ ./abc
Enter array width: 5
Enter mask width (must be odd): 3
Enter 5 elements for array N: 5 15 25 35 45
Enter 3 elements for mask M: 1 2 3

Result P:
55.00 110.00 170.00 230.00 125.00
```

A1.cu

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

__global__ void calculateTotalKernel(float* itemPrices, int* purchaseMatrix, float* individualTotals, int
numFriends, int numItems) {
int friendIdx = blockIdx.x * blockDim.x + threadIdx.x;

if (friendIdx < numFriends) {
float sum = 0.0f;
for (int i = 0; i < numItems; i++) {
```

```
sum += itemPrices[i] * purchaseMatrix[friendIdx * numItems + i];
}
individualTotals[friendIdx] = sum;
}
}

int main() {
int numItems, numFriends;

printf("Enter the number of items available in the mall: ");
scanf("%d", &numItems);

float* h_itemPrices = (float*)malloc(numItems * sizeof(float));
char** itemNames = (char**)malloc(numItems * sizeof(char*));

for (int i = 0; i < numItems; i++) {
itemNames[i] = (char*)malloc(50 * sizeof(char));
printf("Enter name for item %d: ", i + 1);
scanf("%s", itemNames[i]);
printf("Enter price for %s: ", itemNames[i]);
scanf("%f", &h_itemPrices[i]);
}

printf("\n--- SHOPPING MALL MENU ---\n");
for (int i = 0; i < numItems; i++) {
printf("%d. %s - $%.2f\n", i + 1, itemNames[i], h_itemPrices[i]);
}

printf("\nEnter the number of friends: ");
scanf("%d", &numFriends);

int* h_purchaseMatrix = (int*)malloc(numFriends * numItems * sizeof(int));
float* h_individualTotals = (float*)malloc(numFriends * sizeof(float));

for (int f = 0; f < numFriends; f++) {
printf("\nFriend %d, enter quantities for each item:\n", f + 1);
for (int i = 0; i < numItems; i++) {
printf(" Quantity of %s: ", itemNames[i]);
scanf("%d", &h_purchaseMatrix[f * numItems + i]);
}
}

float *d_itemPrices, *d_individualTotals;
int *d_purchaseMatrix;
```

```
cudaMalloc((void**)&d_itemPrices, numItems * sizeof(float));
cudaMalloc((void**)&d_purchaseMatrix, numFriends * numItems * sizeof(int));
cudaMalloc((void**)&d_individualTotals, numFriends * sizeof(float));

cudaMemcpy(d_itemPrices, h_itemPrices, numItems * sizeof(float), cudaMemcpyHostToDevice);
cudaMemcpy(d_purchaseMatrix, h_purchaseMatrix, numFriends * numItems * sizeof(int),
cudaMemcpyHostToDevice);

int threadsPerBlock = 256;
int blocksPerGrid = (numFriends + threadsPerBlock - 1) / threadsPerBlock;

calculateTotalKernel<<<blocksPerGrid, threadsPerBlock>>>(d_itemPrices, d_purchaseMatrix,
d_individualTotals, numFriends, numItems);

cudaMemcpy(h_individualTotals, d_individualTotals, numFriends * sizeof(float),
cudaMemcpyDeviceToHost);

float grandTotal = 0.0f;
printf("\n--- FINAL BILL ---\n");
for (int f = 0; f < numFriends; f++) {
printf("Friend %d Total: $%.2f\n", f + 1, h_individualTotals[f]);
grandTotal += h_individualTotals[f];
}
printf("------------------\n");
printf("Grand Total: $%.2f\n", grandTotal);

cudaFree(d_itemPrices);
cudaFree(d_purchaseMatrix);
cudaFree(d_individualTotals);
for (int i = 0; i < numItems; i++) free(itemNames[i]);
free(itemNames);
free(h_itemPrices);
free(h_purchaseMatrix);
free(h_individualTotals);

return 0;
}
```

Output:

```
6CSEC1@debian:~/Documents/230905152_PPL/L10$ nvcc -o abc a1.cu
6CSEC1@debian:~/Documents/230905152_PPL/L10$ ./abc
Enter the number of items available in the mall: 3
Enter name for item 1: football
Enter price for football: 999
Enter name for item 2: shoes
Enter price for shoes: 1699
Enter name for item 3: books
Enter price for books: 200

--- SHOPPING MALL MENU ---
1. football - $999.00
2. shoes - $1699.00
3. books - $200.00

Enter the number of friends: 2

Friend 1, enter quantities for each item:
  Quantity of football: 1
  Quantity of shoes: 2
  Quantity of books: 1

Friend 2, enter quantities for each item:
  Quantity of football: 2
  Quantity of shoes: 1
  Quantity of books: 2

--- FINAL BILL ---
Friend 1 Total: $4597.00
Friend 2 Total: $4097.00
------------------
Grand Total: $8694.00
```